

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Logic Design with HDL (CO1026)

Assignment Report

Designing a simple RISC Processor

Instructor: Nguyễn Thành Lộc

Students:	Trần Vinh Quang	2550342
	Lê Hiên Vinh	2453422
	Nguyễn Đức Thiên Tân	2550345
	Đặng Ngọc Lan Anh	2550258

HO CHI MINH CITY, JUNE 2026



Contents

I	Introduction	3
I.a	Project Overview	3
I.b	Objectives	3
I.c	Tools and Environment	4
I.d	Main Features of the CPU	4
I.e	Report Organization	5
II	Theoretical Overview	5
II.a	Fetch Phase	6
II.b	Execution Phase	6
III	Design	7
III.a	Top-level System Architecture	7
III.b	Functional Blocks	8
III.b.1	Program Counter (PC)	8
III.b.2	Address Multiplexer (Address MUX)	8
III.b.3	Arithmetic Logic Unit (ALU)	8
III.b.4	Controller Unit	9
III.b.5	Instruction Register (IR)	10
III.b.6	Accumulator Register (AC)	10
III.b.7	Memory	10
IV	Implementation	11
V	Testing and Evaluation	12
VI	Conclusion	12
VII	Conclusion and Future Work	12
VII.a	Future Plan	13
VIII	Member Contribution	14

I Introduction

I.a Project Overview

A Central Processing Unit (CPU) is the core component of a computer system, responsible for fetching instructions, decoding them, executing operations, and controlling data movement between internal registers and memory. In this project, a simple Reduced Instruction Set Computer (RISC) CPU is designed and implemented using *Verilog Hardware Description Language (HDL)*.

The proposed CPU follows a simplified RISC architecture with a compact instruction format. Each instruction consists of a 3-bit opcode and a 5-bit operand field, allowing the processor to support eight basic instructions and address a memory space of 32 locations. Although the architecture is simple, it demonstrates the fundamental principles of processor design, including instruction fetching, instruction decoding, arithmetic and logic execution, memory access, and program control.

The CPU is organized as a set of functional hardware modules, including the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, Arithmetic Logic Unit, and Controller. These modules are designed independently and then integrated into a complete processor system. The CPU operates based on clock and reset signals and continues executing instructions until a halt instruction is encountered.

I.b Objectives

The main objective of this project is to design, implement, and verify a simple RISC CPU at the register-transfer level using Verilog HDL. Through this project, the design process of a basic processor can be studied from both theoretical and practical perspectives.

The specific objectives of the project are as follows:

- To understand the fundamental organization and operation of a RISC-based CPU.
- To design the main functional blocks of the CPU, including the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, ALU, and Controller.
- To implement each functional block using Verilog HDL with a modular and hierarchical design approach.
- To integrate all modules into a complete CPU datapath and control path.
- To verify the correctness of each module through individual testbenches.
- To evaluate the complete CPU by observing simulation waveforms and checking the execution of supported instructions.

By completing these objectives, the project provides practical experience in digital system design, hardware modeling, finite state machine design, and simulation-based verification.

I.c Tools and Environment

The main tool used in this project is Vivado Design Suite. Vivado provides an integrated development environment for designing, simulating, and verifying digital hardware systems described using Verilog HDL. In this project, Vivado is used to create Verilog source files, develop testbenches, run simulations, and analyze signal waveforms. The source code is formatted, linted, and maintained using **Verible** and the SystemVerilog Language Server (SVLS) to ensure strict hardware coding conventions.

Each CPU component is implemented as an independent Verilog module and verified using a corresponding testbench. A custom **Python** script is implemented with Icarus Verilog (**iverilog**) serves as the primary compiler for rapid, iterative command-line simulations and unit testing. After unit-level verification, the modules are connected together in a top-level design to perform system-level simulation. The waveform viewer in Vivado is used to observe the behavior of internal signals, such as the program counter value, memory address, instruction register output, accumulator value, ALU result, and controller signals.

Vivado is suitable for this project because it supports hierarchical hardware design, Verilog-based modeling, testbench simulation, and detailed waveform analysis. These features allow the design to be verified systematically before further hardware synthesis or implementation.

I.d Main Features of the CPU

The designed CPU supports a small but representative set of instructions. Since the opcode field is 3 bits wide, the CPU can define up to 8 instructions. These instructions include program control, arithmetic operations, logic operations, memory access, and conditional execution.

The main features of the CPU are summarized as follows:

- A 3-bit opcode field supporting eight basic instructions.
- A 5-bit operand field supporting 32 memory address locations.
- A Program Counter used to store and update the current instruction address.
- An Address Multiplexer used to select between the instruction address and operand address.
- A Memory module used to store both instructions and data.
- An Instruction Register used to hold and decode the current instruction.

- An Accumulator Register used to store intermediate and final computation results.
- An ALU used to perform arithmetic and logic operations.
- A Controller implemented as a finite state machine to generate control signals for instruction execution.
- A halt mechanism that stops program execution when the halt instruction is detected.

The CPU performs its operation through a sequence of basic stages: instruction fetch, instruction decode, operand access, execution, and result storage. This execution flow reflects the essential behavior of a real processor while remaining simple enough for analysis, implementation, and verification.

I.e Report Organization

This report is organized into six main sections.

Section I introduces the project, including the project overview, objectives, tools and environment, main CPU features, and report structure.

Section II presents the theoretical background of the project. It explains the basic concepts of RISC architecture, instruction format, CPU execution cycle, datapath, control path, and supported instructions.

Section III describes the design of the CPU. This section presents the overall system architecture, block diagram, and detailed functions of each hardware module.

Section IV discusses the implementation of the CPU using Verilog HDL. It explains the module structure, implementation approach, controller finite state machine, memory organization, and top-level integration.

Section V presents the testing and evaluation process. It includes unit-level testbenches, system-level simulation, waveform analysis, and functional evaluation of the CPU.

Section VI concludes the report by summarizing the achieved results, discussing difficulties encountered during the project, and proposing possible future improvements.

II Theoretical Overview

Based on the project specification, the RISC CPU designed in this assignment features a 3-bit opcode and a 5-bit operand. This architecture supports 8 types of instructions (HLT, SKZ, ADD, AND, XOR, LDA, STO, JMP) and a 32-word memory address space. The processor operates synchronously with a clock signal and an active-high reset signal (the default reset state is INST_ADDR). Program execution stops upon encountering the HLT instruction.



By analyzing the controller state table and the functional requirements of each block, the CPU's operation flow can be divided into two primary phases spanning 8 clock cycles: the Fetch Phase and the Execution Phase.

Table II.1: CPU Controller state and Control signals

Outputs	Phase								Notes
	INST_ADDR	INST_FETCH	INST_LOAD	IDLE	OP_ADDR	OP_FETCH	ALU_OP	STORE	
sel	1	1	1	1	0	0	0	0	ALU OP = 1 if opcode is ADD, AND, XOR or LDA
rd	0	1	1	1	0	ALUOP	ALUOP	ALUOP	
ld_ir	0	0	1	1	0	0	0	0	
halt	0	0	0	0	HALT	0	0	0	
inc_pc	0	0	0	0	1	0	SKZ && zero	0	
ld_ac	0	0	0	0	0	0	0	ALUOP	
ld_pc	0	0	0	0	0	0	JMP	JMP	
wr	0	0	0	0	0	0	0	STO	
data_e	0	0	0	0	0	0	STO	STO	

II.a Fetch Phase

- **State 1: INST_ADDR.** The Program Counter (PC) provides the address of the current instruction. The Address Mux selects the PC address (**sel** = 1) to access the memory.
- **State 2: INST_FETCH.** The memory read signal (**rd** = 1) is asserted. The instruction data is retrieved from Memory.
- **State 3: INST_LOAD.** The fetched instruction is loaded into the Instruction Register (IR) via the **ld_ir** = 1 control signal.
- **State 4: IDLE.** An intermediate wait state for the system to stabilize.

II.b Execution Phase

- **State 5: OP_ADDR.** The IR splits the instruction into a 3-bit Opcode and a 5-bit Operand Address. The Address Mux switches to select the operand address from the IR (**sel** = 0). Simultaneously, the PC increments (**inc_pc** = 1) to prepare for the next instruction cycle.

- **State 6: OP_FETCH.** If the instruction requires data from memory (e.g., ADD, AND, LDA), the CPU reads the data from the operand address in the Memory ($rd = 1$).
- **State 7: ALU_OP.** The Arithmetic Logic Unit (ALU) performs the required computation between the data fetched from Memory and the data currently held in the Accumulator (AC), based on the decoded Opcode. If the instruction is SKZ and the **zero** flag condition is met, the PC increments once more ($inc_pc = 1$) to skip the subsequent instruction.
- **State 8: STORE.** The result of the operation is either stored back into the Memory (for the ST0 instruction, asserting $wr = 1$ and $data_e = 1$) or loaded into the Accumulator (for arithmetic and logic instructions, asserting $ld_ac = 1$).

III Design

III.a Top-level System Architecture

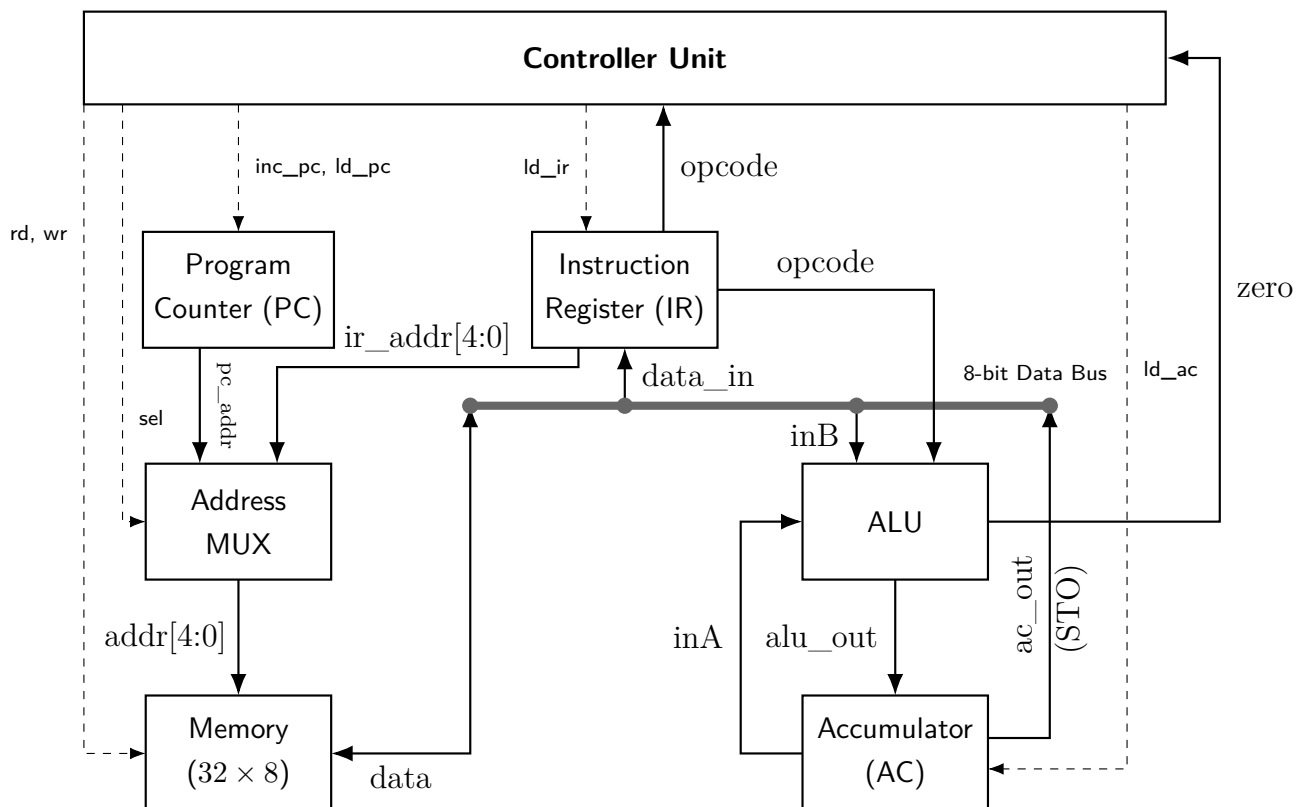


Figure III.1: Top-Level Architecture of the RISC Processor



III.b Functional Blocks

III.b.1 Program Counter (PC)

- **Function:**
 - The counter is a crucial component used to track and count program instructions.
 - It can also be utilized to count program states.
 - The counter operates synchronously on the rising edge of the `clk` signal.
 - The `reset` signal is active-high, clearing the counter back to 0.
 - The counter has a width of 32 bits.
 - It features a load function to force an arbitrary address into the counter. If not loading, the counter increments normally.
- **Inputs:** `clk`, `rst`, `ld_pc`, `inc_pc`, `data_in`.
- **Outputs:** `pc_out`.

III.b.2 Address Multiplexer (Address MUX)

- **Function:**
 - The Address MUX selects between the instruction address (during the instruction fetch phase) and the operand address (during the instruction execution phase).
 - The MUX has a default data width of 32 bits.
 - The width is parameterized to allow for easy modification if necessary.
- **Inputs:** `pc_addr`, `ir_addr`, `sel`.
- **Outputs:** `addr_out`.

III.b.3 Arithmetic Logic Unit (ALU)

- **Function:**
 - The ALU executes arithmetic and logical operations. The specific operation performed depends on the instruction's opcode.
 - The ALU executes 8 distinct operations on 32-bit operands (`inA` and `inB`).
 - It produces a 32-bit output and a 1-bit asynchronous `zero` flag.
 - The `zero` flag indicates whether the input `inA` is equal to 0.

- **Inputs:** inA, inB, opcode.
- **Outputs:** alu_out, zero.

Table III.1: ALU Opcode Lookup Table

Opcode	Code	Operation Description	Output
HLT	000	Halts program execution.	inA
SKZ	001	Checks if the ALU result is equal to 0. If zero, it skips the next instruction; otherwise, execution continues normally.	inA
ADD	010	Adds the value in the Accumulator to the memory value at the instruction's address, returning the result to the Accumulator.	inA + inB
AND	011	Performs a bitwise AND on the Accumulator value and the memory value at the instruction's address.	inA AND inB
XOR	100	Performs a bitwise XOR on the Accumulator value and the memory value at the instruction's address.	inA XOR inB
LDA	101	Reads the value from the address in the instruction and loads it into the Accumulator.	inB
STO	110	Writes the Accumulator's data to the memory address specified in the instruction.	inA
JMP	111	Unconditional jump; jumps to the target address in the instruction and continues execution.	inA

III.b.4 Controller Unit

- **Function:** The central finite state machine that decodes instructions and dictates the operation of all other blocks via control signals.
- **Inputs:** clk, rst, opcode, zero.
- **Outputs:** sel, rd, ld_ir, halt, inc_pc, ld_ac, ld_pc, wr, data_e.

Table III.2: Control Execution Scenarios by Opcode

Instruction Scenario	Active States	Expected Control Signals
HLT (000)	OP_ADDR	halt = 1
ADD, AND, XOR, LDA	OP_FETCH to STORE	rd = 1, ld_ac = 1 (at STORE phase)
STO (110)	ALU_OP, STORE	wr = 1, data_e = 1
JMP (111)	ALU_OP, STORE	ld_pc = 1

III.b.5 Instruction Register (IR)

- **Function:** As described in the reference documentation, it captures and holds the current instruction fetched from memory, splitting it into an opcode and an operand.
- **Inputs:** clk, rst, ld_ir, data_in.
- **Outputs:** opcode, operand.

III.b.6 Accumulator Register (AC)

- **Function:** As described in the reference documentation, it acts as the primary working register for ALU operations and data transfers.
- **Inputs:** clk, rst, ld_ac, data_in.
- **Outputs:** ac_out.

III.b.7 Memory

- **Function:**
 - The memory module stores both instructions and data.
 - It is designed to separate read and write functions while utilizing a single bidirectional data port. Reading and writing cannot occur simultaneously.
 - Configured for 32-bit addresses and 32-bit data.
 - Uses 1-bit control signals to enable reading (**rd**) and writing (**wr**).
 - Memory operations are synchronized to the rising edge of the clk signal.
- **Inputs:** clk, rd, wr, addr.
- **Inout:** data.

IV Implementation

Listing IV.1: ALU Module

```
1 module ALU(  
2     input [2:0] opcode,  
3     input [7:0] inA,  
4     input [7:0] inB,  
5  
6     output reg [7:0] alu_out,  
7     output zero  
8 );  
9  
10 assign zero = (inA == 8'b00000000);  
11  
12 always @(*) begin  
13     case(opcode)  
14  
15         3'b000: alu_out = inA;           // HLT  
16         3'b001: alu_out = inA;           // SKZ  
17         3'b010: alu_out = inA + inB;     // ADD  
18         3'b011: alu_out = inA & inB;     // AND  
19         3'b100: alu_out = inA ^ inB;     // XOR  
20         3'b101: alu_out = inB;           // LDA  
21         3'b110: alu_out = inA;           // STO  
22         3'b111: alu_out = inA;           // JMP  
23  
24         default: alu_out = 8'b00000000;  
25  
26     endcase  
27 end  
28  
29 endmodule
```



V Testing and Evaluation

VI Conclusion

VII Conclusion and Future Work

In this assignment, our group designed and implemented a simple RISC processor using Verilog HDL. The processor follows an accumulator-based architecture and uses an 8-bit instruction format, consisting of a 3-bit opcode and a 5-bit operand field. Based on this format, the CPU supports eight instructions: HLT, SKZ, ADD, AND, XOR, LDA, STO, and JMP. The design also includes a 5-bit program counter and a 32-location memory space, which are consistent with the project specification.

The processor was developed using a modular design approach. Core components, including the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, ALU, and Controller, were implemented as independent Verilog modules and then integrated into the top-level CPU module. This organization made the datapath and control path clearer, while also making simulation, debugging, and system integration more manageable.

A key part of the design was the Controller, which was implemented as a finite state machine with eight states: INST_ADDR, INST_FETCH, INST_LOAD, IDLE, OP_ADDR, OP_FETCH, ALU_OP, and STORE. These states coordinate the main phases of instruction execution, including instruction fetch, decoding, operand access, ALU operation, accumulator update, memory write-back, program counter update, and halt control. Through simulation, the processor was verified to execute the supported instructions correctly at both module and system levels.

The main strength of the design is its clear separation between datapath components and control logic. Since each module was tested independently before being connected in the complete CPU, functional errors were easier to identify and correct. In addition, the use of both module-level and CPU-level testbenches helped confirm the correctness of individual blocks as well as the overall instruction execution flow.

However, the current processor still has several limitations. The instruction set is relatively small and does not support more complex arithmetic or comparison operations. The 5-bit address bus also limits the memory space to only 32 locations, which restricts the size of executable programs. Moreover, the controller follows a fixed multi-state execution sequence, meaning that some simple instructions may require more clock cycles than necessary. The project also focuses mainly on functional simulation, while synthesis results, timing delay, resource utilization, and power consumption have not yet been analyzed in detail.

VII.a Future Plan

Based on these limitations, several improvements can be considered for future development. First, the instruction set can be extended to improve the flexibility of the processor. Additional instructions such as SUB, MUL, DIV, shift operations, and comparison instructions would allow the CPU to support more diverse computational tasks. For example, comparison and shift instructions would make the processor more capable of handling conditional operations and bit-level data manipulation.

Second, the memory system can be expanded by increasing the address width. Since the current 5-bit address bus only supports 32 memory locations, a wider address bus would allow the processor to access a larger program and data memory space. This improvement would make it possible to execute longer instruction sequences and store more intermediate data during program execution.

Third, the Controller can be optimized to reduce unnecessary execution cycles. In the current design, instructions follow a fixed sequence of states, even when some operations do not require all stages. Future versions could use instruction-dependent state transitions so that simpler instructions, such as HLT or JMP, complete in fewer cycles. More advanced control techniques, including pipelining and hazard handling, could also be explored to improve instruction throughput.

Fourth, verification should be extended with more comprehensive test programs. Although module-level and CPU-level simulations were performed, additional cases should be tested, such as repeated jump operations, consecutive store instructions, SKZ behavior when the accumulator is zero and non-zero, and boundary memory access at the lowest and highest addresses. These cases would help evaluate the robustness of the processor under less straightforward execution scenarios.

Finally, synthesis and implementation analysis should be performed to evaluate the hardware cost of the design. Resource utilization, critical path delay, maximum operating frequency, and power consumption should be analyzed after synthesis. This would provide a more complete understanding of the processor not only as a functional Verilog model, but also as a hardware design that could be implemented on an FPGA or similar digital platform.

Overall, this assignment helped us understand the fundamental principles of processor design, especially the interaction between datapath modules and control logic. It also provided practical experience in using Verilog HDL, designing a finite state machine controller, integrating multiple hardware modules, and verifying a digital system through simulation.



VIII Member Contribution

Table VIII.1: Member Contribution

No.	Member	Student ID	Contribution
1	Trần Vinh Quang	2550342	Implemented the Controller module, including the finite state machine and control signal generation. Contributed to the project report.
2	Lê Hiền Vinh	2453422	Implemented the Program Counter and top-level CPU modules. Developed the testbenches and expected output files, and performed simulation verification for the system.
3	Nguyễn Đức Thiên Tân	2550345	Implemented the Memory and Instruction Register modules, including memory access logic and instruction decoding. Contributed to the project report.
4	Đặng Ngọc Lan Anh	2550258	Implemented the ALU and Accumulator modules, including arithmetic, logic, zero-flag, and register operations. Contributed to the project report.